

Exercícios de autorrevisão

17.1 Preencha os espaços em cada uma das sentenças:

- a) Os membros de classe são acessados por meio do operador _____ em conjunto com o nome de um objeto (ou referência a um objeto) da classe ou por meio do operador _____ em conjunto com um ponteiro de um objeto da classe.
- b) Os membros de classe especificados como _____ são acessíveis apenas às funções-membro da classe e às amigas da classe.
- c) Os membros da classe especificados como _____ são acessíveis em qualquer lugar em que um objeto da classe esteja no escopo.
- d) A _____ pode ser usada para atribuir um objeto de uma classe a outro objeto da mesma classe.

17.2 Encontre o(s) erro(s) em cada um dos itens a seguir e explique como corrigi-lo(s):

- a) Suponha que o protótipo a seguir esteja declarado na classe `Time`:

```
void ~Time( int );
```

- b) A seguir está uma definição parcial da classe `Time`:

```
class Time
{
public:
    // protótipos de função
private:
    int hour = 0;
    int minute = 0;
    int second = 0;
}; // fim da classe Time
```

- c) Suponha que o protótipo a seguir seja declarado na classe `Employee`:

```
int Employee( string, string );
```

Respostas dos exercícios de autorrevisão

17.1 a) ponto (`.`), seta (`->`). b) `private`. c) `public`. d) Atribuição de membro default (realizado pelo operador de atribuição).

17.2 a) *Erro*: Destrutores não podem retornar valores (nem especificar um valor de retorno) ou usar argumentos.

Correção: Remova o tipo de retorno `void` e o parâmetro `int` da declaração.

b) *Erro*: Os membros não podem ser inicializados explicitamente na definição da classe.

Correção: Remova a inicialização explícita da definição de classe e inicialize os dados-membro em um construtor.

c) *Erro*: Os construtores não podem retornar valores.

Correção: Remova o tipo de retorno `int` da declaração.

Exercícios

17.3 Qual é a finalidade do operador de resolução de escopo?

17.4 *Aperfeiçoamento da classe `Time`*. Forneça um construtor que seja capaz de usar a hora atual das funções `time` e `localtime` — declaradas no cabeçalho `<ctime>` da biblioteca-padrão de C++ — para inicializar um objeto da classe `Time`.

17.5 *Classe `Complex`*. Crie uma classe chamada `Complex` para realizar aritmética com números complexos. Escreva um programa para testar sua classe. Números complexos têm a forma

$$\text{parteReal} + \text{parteImaginária} * i$$

onde i é

$$\sqrt{-1}$$

Use variáveis `double` para representar os dados `private` da classe. Forneça um construtor que permita que um objeto dessa classe seja inicializando quando declarado. O construtor deverá conter valores default caso nenhum inicializador seja fornecido. Forneça funções-membro `public` que realizem as seguintes tarefas:

- a) Somar dois números `Complex`: as partes reais são somadas e as partes imaginárias são somadas.
- b) Subtrair dois números `Complex`: a parte real do operando da direita é subtraída da parte real do operando da esquerda, e a parte imaginária do operando da direita é subtraída da parte imaginária do operando da esquerda.
- c) Imprimir números `Complex` na forma (a, b) , onde a é a parte real e b é a parte imaginária.

17.6 Classe Rational. Crie uma classe chamada `Rational` para realizar aritmética com frações. Escreva um programa para testar sua classe.

Use variáveis inteiras para representar os dados `private` da classe — o numerador e o denominador. Forneça um construtor que permita que um objeto dessa classe seja inicializado quando for declarado. O construtor deverá conter valores default para o caso de nenhum inicializador ser fornecido, e deverá armazenar a fração no formato reduzido. Por exemplo, a fração

$$\frac{2}{4}$$

seria armazenada no objeto como 1 no numerador e 2 no denominador. Forneça funções-membro `public` que realizem cada uma das tarefas a seguir:

- a) Somar dois números `Rational`. O resultado deverá ser armazenado na forma reduzida.
- b) Subtrair dois números `Rational`. O resultado deverá ser armazenado na forma reduzida.
- c) Multiplicar dois números `Rational`. O resultado deverá ser armazenado na forma reduzida.
- d) Dividir dois números `Rational`. O resultado deverá ser armazenado na forma reduzida.
- e) Imprimir números `Rational` no formato `a/b`, onde `a` é o numerador e `b` é o denominador.
- f) Imprimir números `Rational` no formato de ponto flutuante.

17.7 Aperfeiçoamento da classe Time. Modifique a classe `Time` das figuras 17.8 e 17.9 para incluir uma função-membro `tick` que incremente a hora armazenada em um objeto `Time` em um segundo. O objeto `Time` sempre deverá permanecer em um estado coerente. Escreva um programa que teste a função-membro `tick` em um loop que imprima a hora no formato-padrão durante cada iteração do loop para mostrar que a função-membro `tick` funciona corretamente. Não se esqueça dos testes para:

- a) Incrementar para o próximo minuto.
- b) Incrementar para a próxima hora.
- c) Incrementar para o próximo dia (ou seja, de 11:59:59 PM a 12:00:00 AM).

17.8 Aperfeiçoamento da classe Date. Modifique a classe `Date` das figuras 17.17 e 17.18 para fazer a verificação de erro nos valores inicializadores dos dados-membro `month`, `day` e `year`. Além disso, forneça uma função-membro `nextDay` para incrementar o dia em um. O objeto `Date` sempre deverá permanecer em um estado coerente. Escreva um programa que teste a função `nextDay` em um loop que imprima a data durante cada iteração,

para ilustrar que `nextDay` funciona corretamente. Não se esqueça dos testes para:

- a) Incrementar para o próximo mês.
- b) Incrementar para o próximo ano.

17.9 Combinação das classes Time e Date. Combine a classe `Time` modificada do Exercício 17.7 e a classe `Date` modificada do Exercício 17.8 em uma classe chamada `DateAndTime`. (No Capítulo 20, discutiremos a herança, que nos permite realizar essa tarefa rapidamente, sem modificar as definições de classe existentes.) Modifique a função `tick` para chamar a função `nextDay` se a hora incrementar para o dia seguinte. Modifique as funções `printStandard` e `printUniversal` para mostrar a data e a hora. Escreva um programa para testar a nova classe `DateAndTime`. Especificamente, teste o incremento da hora para o dia seguinte.

17.10 Retorno dos indicadores de erro das funções set da classe Time. Modifique as funções `set` na classe `Time` das Figuras 17.8 a 17.9 para retornar valores de erro apropriados se for feita uma tentativa de *definir* um dado-membro de um objeto da classe `Time` para um valor inválido. Escreva um programa que teste sua nova versão da classe `Time`. Mostre mensagens de erro quando as funções `set` retornam valores de erro.

17.11 Classe Retangulo. Crie uma classe `Retangulo` com atributos `comprimento` e `largura`, cada uma com o valor default 1. Forneça funções-membro que calculem o perímetro e a área do retângulo. Além disso, forneça funções `set` e `get` para os atributos `comprimento` e `largura`. As funções `set` devem verificar se `comprimento` e `largura` são números em ponto flutuante maiores que 0.0 e menores que 20.0.

17.12 Aperfeiçoamento da classe Retangulo. Crie uma classe `Retangulo` mais sofisticada do que aquela criada no Exercício 17.11. Essa classe armazena apenas as coordenadas cartesianas dos quatro cantos do retângulo. O construtor chama uma função `set`, que aceita quatro conjuntos de coordenadas e verifica se cada uma delas está no primeiro quadrante sem coordenadas x ou y isoladas maiores que 20.0. A função `set` também verifica se as coordenadas fornecidas realmente especificam um retângulo. Forneça funções-membro que calculem `comprimento`, `largura`, `perimetro` e `area`. O `comprimento` é a maior das duas dimensões. Inclua uma função predado `quadrado` que determine se o retângulo é um quadrado.

17.13 Aperfeiçoamento da classe Retangulo. Modifique a classe `Retangulo` do Exercício 17.12 para incluir uma função `desenhar` que mostre o retângulo dentro de uma caixa de 25 por 25 delimitando a parte do primeiro qua-

drante em que o retângulo reside. Inclua uma função `setFillCharacter` que especifique o caractere do qual o corpo do retângulo será desenhado. Inclua uma função `setPerimeterCharacter` que especifique o caractere que será usado para desenhar a borda do retângulo. Se você for ambicioso, poderá incluir funções para redimensionar o tamanho do retângulo, girá-lo e movê-lo dentro da parte designada do primeiro quadrante.

17.14 Classe *HugeInteger*. Crie uma classe `HugeInteger` que use um array de 40 elementos de dígitos para armazenar inteiros com até 40 dígitos cada. Forneça funções-membro `input`, `output`, `add` e `subtract`. Para comparar objetos `HugeInteger`, forneça funções `isEqualTo`, `isNotEqualTo`, `isGreaterThan`, `isLessThan`, `isGreaterThanOrEqualTo` e `isLessThanOrEqualTo` — cada uma delas é uma função ‘predicado’ que simplesmente retorna `true` se o relacionamento for mantido entre os dois `HugeIntegers`, e retorna `false` se o relacionamento não for mantido. Além disso, forneça uma função predicado `isZero`. Se você for ambicioso, forneça funções-membro `multiply`, `divide` e `modulus`.

17.15 Classe *Velha*. Crie uma classe `Velha` que lhe permitirá escrever um programa completo para jogar o jogo da velha. A classe contém como dados `private` um array bidimensional de 3 por 3 com inteiros. O construtor deverá inicializar o tabuleiro vazio com zeros. Permita que duas pessoas joguem. Sempre que o primeiro jogador fizer sua jogada, coloque um 1 no quadrado especificado. Coloque um 2 para a jogada do segundo jogador. Cada movimento deverá estar em um quadrado vazio. Depois de cada movimento, determine se o jogo foi ganho ou se está empatado. Se você for ambicioso, modifique seu programa de modo que o computador jogue no lugar de um dos jogadores. Além disso, permita que o jogador especifique se ele ou ela deseja ser o primeiro ou o segundo. Se você for excepcionalmente ambicioso, desenvolva um programa para um jogo da velha tridimensional em um tabuleiro de 4 por 4 por 4. [*Cuidado*: este é um projeto extremamente desafiador, que poderá exigir muitas semanas de esforço!]